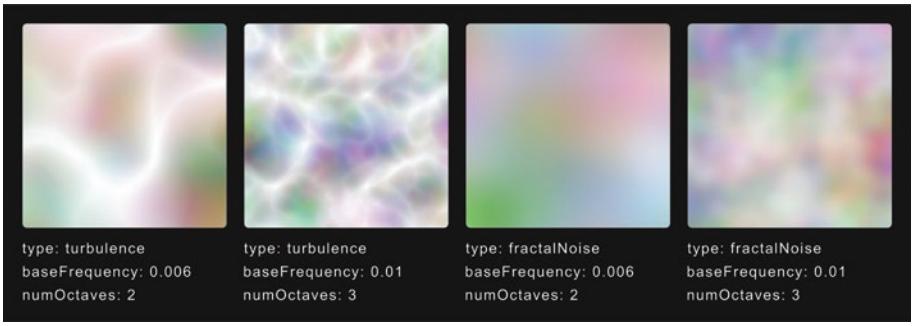

Figure 8-7. *Varying the base frequency and octaves for the two noise types*

The best way to get a feel for how `feTurbulence` works is to tweak an interactive example, so I've created a pen for you here for this purpose: `davidmatthew.ie/generative-art-javascript-svg#turbulence`. I prefer working with `baseFrequency` values at the very low end of spectrum and modifying the `seed` to select a pattern at random; this can produce a huge variety of interesting textures. Very high `baseFrequency` values can be useful however if you want to simulate something more granular, like the classic film grain effect or a sandpaper-like texture.

Displacement

One filter primitive that works particularly well alongside `feTurbulence` is `feDisplacementMap`. This primitive allows us to distort the first input, `in`, using values from the second input, `in2`. This second input acts as the map from which we can derive the displacement values, that is, the values that decide how much distortion we should apply along the x and y axes. This means that we could use the output from `feTurbulence` to distort our `SourceGraphic` by feeding both through `feDisplacementMap`.

Besides the aforementioned `in` and `in2`, the `feDisplacementMap` primitive has three other attributes: the first is `scale`, which determines how much distortion to apply (i.e., to what extent is `in` distorted by `in2`).

The other two attributes, `xChannelSelector` and `yChannelSelector`, allow us to specify which channels are responsible for horizontal and vertical displacement. They accept values of `R`, `G`, `B`, or `A`, representing the three color channels along with the alpha channel.

If we take a single color channel, `R`, as an example, its value can range from 0 to 255. Values below 127 result in negative displacement (they're shifted left along the x axis, or up along the y axis), whereas values above 127 result in positive displacement (shifted to the right along the x axis, or down along the y axis).

In Figure 8-8, the output of an `feTurbulence` primitive (center) is used as the map to displace the source graphic (left), resulting in some soft wave-like distortion (right).

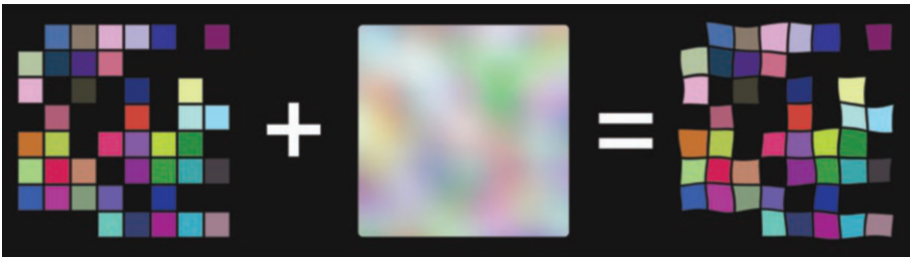

Figure 8-8. *Using turbulence as a displacement map source*

Creating a Cosmic Bubble

This next example will piece together several of the filter primitives we've covered so far and apply it to the simplest of source graphics: the circle. Our aim will be to create something a little "cosmic" by compositing together some turbulence, blur, and distortion.

First things first, make a copy of our template folder and rename it to `22-hubble-bubble` (or another name of your choice). In the usual spot below the background, let's set up our source graphic in the center of the `viewBox`, randomizing its radius a little.

```
// Create the source graphic.
svg.create('circle').set({
  cx: 500,
  cy: 500,
  r: Gen.random(250, 350),
  fill: '#000',
  filter: 'url(#cosmic)'
});
```

You'll notice we've referenced our filter already but haven't yet created it. Normally the order of our created elements matters; if we create one circle and then another, the latter will be rendered "above" the former in a layer-like manner. But as filters are created within a `defs` element, it's ok to define them after the element to which they apply. To recap, anything that lives inside the `defs` element isn't directly rendered but must be referenced.

In the next step, we'll create the filter (ensuring the `id` matches the reference to `#cosmic`), and add the first filter primitive to the chain: `feTurbulence`. We'll randomize the seed, keep the dial low on the `baseFrequency`, and crank up the `numOctaves` to `4`, to bring out some of the texture details.

```
// Initialise the filter.
let filter = svg.createFilter('cosmic');

// Create a random amount of turbulence.
filter.create('feTurbulence').set({
  type: 'fractalNoise',
  baseFrequency: Gen.random(0.002, 0.006, true),
  seed: Gen.random(0, 10000),
  numOctaves: 4,
  stitchTiles: 'stitch',
  result: 'turbulence'
});
```

Like `feFlood`, `feTurbulence` is a generator primitive that doesn't require any input; it generates its own output directly. The result of this is that once we initialize an `feTurbulence` primitive as the latest (or only) link in the filter chain, it will simply flood the filter region with its output (which is what you'll see if you run the code at this point). We'll use `feComposite` to fix this later.

In our next step, we'll create a target region to displace (i.e., apply distortion to). To create this target area, what we'll do is soften the edges of the source graphic (the circle) using an `feGaussianBlur`; this has the effect of introducing some transparency to the circle's perimeter. This area of receding transparency (another way of describing a blur) will be what we use to apply distortion to our turbulence. Figure 8-8 shows the result of using turbulence as a displacement map source; this time around we'll be doing the reverse and using turbulence as the displacement map destination. The blurred area of our circle will instead be our displacement map source. Here's how to set up this up (with some randomness mixed in).

```
// Blur the edges of the source graphic.
filter.create('feGaussianBlur').set({
  stdDeviation: Gen.random(10, 25), in: 'SourceGraphic',
  result: 'blurred'
});

// Displace the turbulence with the blurred edge of the circle.
filter.create('feDisplacementMap').set({
  in: 'turbulence',
  in2: 'blurred',
  scale: Gen.random(250, 500),
  result: 'distortion'
});
```

This results in a rim of displaced noise, as shown in Figure 8-9. Note that we've omitted the `xChannelSelector` and `yChannelSelector` attributes of `feDisplacementMap`; these both default to the alpha channel, which is fine in our case.

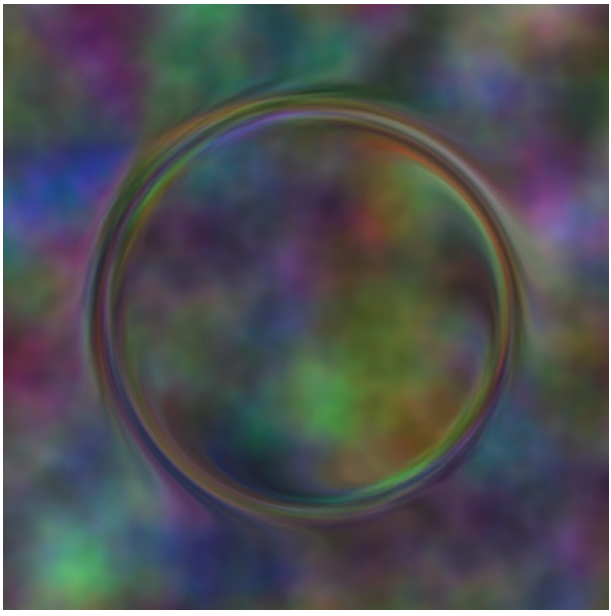

Figure 8-9. *Displacement using the blurred edge of a circle*

We're not fully done yet; as a final step, I want to cut away everything beyond the perimeter of the blurred region. The solution to this is to composite together the outputs of `feDisplacementMap` and `feGaussianBlur` using the `atop` operator. We should then have something that resembles Figure 8-10, our Hubble Bubble.

```
// Remove everything beyond the blurred perimeter.
filter.create('feComposite').set({
  in: 'distortion',
  in2: 'blurred',
  operator: 'atop'
});
```

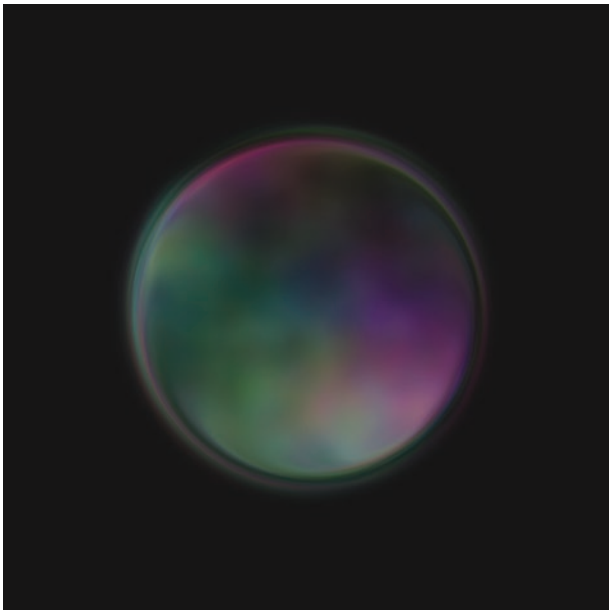

Figure 8-10. *One possible iteration of the Hubble Bubble sketch*

Lighting and Texture

Despite SVG being, by definition, a format for two-dimensional vector graphics, among its filter primitives, you'll find two lighting effects: `feDiffuseLighting` and `feSpecularLighting`. Normally the domain of three-dimensional graphics, lighting typically requires a z-axis component to operate so that surfaces or other objects that protrude in space, that is, have depth, can catch the light cast by the source. SVG coordinate space is devoid of any such third dimension or z axis, so how exactly does lighting function in such an environment?

The answer is that the z axis is simulated by using "bump" maps. Specifically, `feDiffuseLighting` and `feSpecularLighting` both interpret the alpha channel of their connected inputs as a bump map source. A bump map is just what it sounds like, a map of bumps, meaning any raised regions of a surface, like slopes, peaks, folds, and wrinkles.

The challenge with these lighting primitives is knowing how to supply them with nicely gradiented bump maps; anything with very sharp transitions or a lot of areas of flat color can often appear unpleasantly pixelated, or entirely unaffected. In Figure 8-11, we can see the rather stark difference between a distant light source shining on a circle with a radial gradient (left) vs. a circle of the same size with a uniform fill (right).

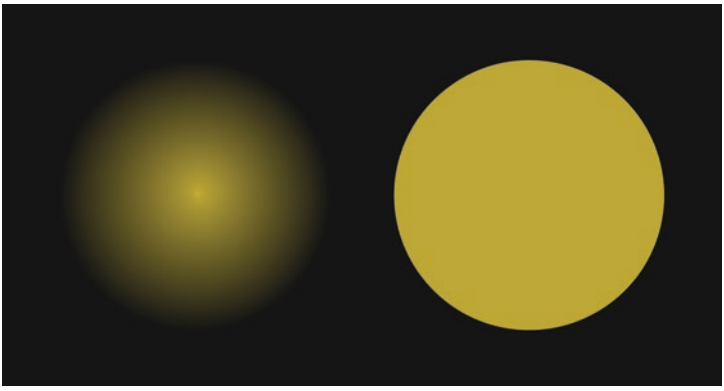

Figure 8-11. *Lighting cast on a radial gradient vs. a flat fill*

Lighting is one of those things in SVG that's very easy to get wrong. When it does go awry, the results often look cheap, choppy, and jagged and can put you off playing with lighting altogether. We need to remember that with filters, we've moved from the realm of vectors to that of pixels, where resolution matters. A more finely gradiented alpha channel will translate to a higher-resolution bump map.

Fortunately, `feTurbulence` is a first-rate source of gradiented alpha channel data. In our next sketch, we'll use `feTurbulence` along with `feDiffuseLighting` and `feDisplacementMap` to simulate a rough, ripped paper effect. But first, we need to delve a little more into the details of the two aforementioned lighting primitives.

Diffuse and Specular Lighting

The `feDiffuseLighting` primitive simulates diffuse reflection. What this means is that the light that strikes the surface of a diffuse-lit object is scattered in all directions, as is characteristic of rough surface textures (like linen). The `feSpecularLighting` primitive, on the other hand, simulates specular reflection, which is characteristic of smoother surfaces (like a bowling ball). With specular reflection, the light that strikes the surface bounces at a definite angle. Figure 8-12 shows the difference between a point light emitted by `feDiffuseLighting` (left) vs. the same light emitted by `feSpecularLighting` (right).

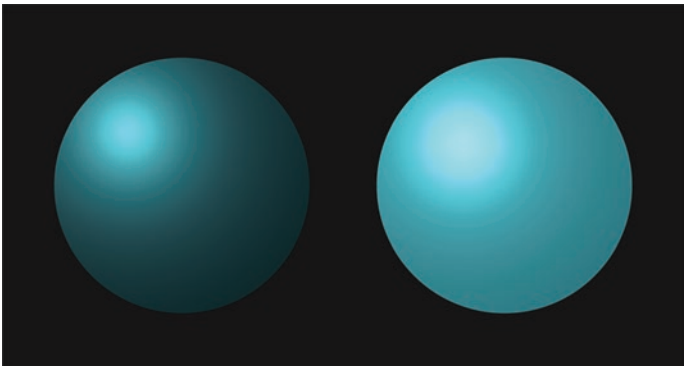

Figure 8-12. *Diffuse vs. specular lighting*

Both lighting primitives take a single input `in` as their bump map source and a `surfaceScale` attribute, which defines the relative height of the bump map. The color of the light can also be customized using the `lighting-color` attribute, which is white by default. Both primitives also share an attribute that defines a constant, `diffuseConstant` in the case of `feDiffuseLighting` and `specularConstant` in the case of `feSpecularLighting`. This constant determines the strength of the light cast by the source. The `feSpecularLighting` primitive boasts an additional

attribute called `specularExponent`, which controls the focal point strength (in Figure 8-12, this would determine the size of the point light's reflection).

Light Sources

I've mentioned distant lights and point lights a couple of times now but haven't explained them. These refer to the type of light emitted by either an `feDiffuseLighting` or `feSpecularLighting` primitive. By themselves, these primitives do not emit any light; they require an extra nested node to define the source. In other words, setting up a light in SVG involves defining two elements: one, the outer node, which defines whether the target surface of the lighting is diffuse or specular; and two, the inner node, which defines the type of light source to be used. Here's how an example might look in raw SVG:

```
<feDiffuseLighting in="SourceGraphic" surfaceScale="1.5">
  <feDistantLight azimuth="330" elevation="45" />
</feDiffuseLighting>
```

There are three light source nodes: `feDistantLight`, an ambient light that emits rays uniformly in all directions (such as the sun); `fePointLight`, a more proximate light source that has a defined focal point; and `feSpotLight`, a directional light that emits its rays through a conical shape, the dimensions of which can be customized. Addressing all three light sources would exceed the scope of this chapter, so we'll stick to the first two: `feDistantLight` and `fePointLight`.

The `feDistantLight` source has two attributes: the `azimuth`, which specifies the angle of direction of the light, and the `elevation`, which specifies the height of the light source. Both are expressed in terms of degrees. To use the sun analogy, at dawn it would rise in the east (at an `azimuth` and `elevation` of `0`) and by midday at the equator, it would have climbed to an `elevation` of `90`, directly overhead.

The fePointLight source has no azimuth or elevation; it is instead defined entirely by x, y, and z attributes. The z attribute would correspond with the elevation or height of the light source, and the x and y attributes refer to the standard two-dimensional position in the current coordinate space.

Simulating Textures

Let's start a new sketch and flesh out some of the theory mentioned previously. Copy our template folder and rename it to 23-rough-paper, and in the usual spot below the background, create a parchment-colored gradient that we'll apply to our paper. The paper itself will be a compound path consisting of two shapes (the result of ripping it in two). As you can see, it's possible to extend paths beyond a terminating Z command by simply adding a new M command (the equivalent of lifting your pen to start a new line elsewhere). Once we have the ripped paper set up, we center it.

```
// Create a parchment-coloured gradient.
svg.createGradient('parchment', 'linear', ['#fffbeb',
'#fde68a'], 90);

// Create the path for the ripped paper.
let paper = svg.create('path').set({
  fill: 'url(#parchment)',
  stroke: '#4444',
  d: 'M 0,0 h 175 l 175,550 h -350 Z M 210,0 h 340 v
  550 h -165 Z',
  filter: 'url(#rough-paper)'
});

// Centre it.
paper.moveTo(500, 500);
```

The composition we're keeping deliberately simple so that we can give more focus to the filter effect. We'll initialize this next and add the first primitive to the chain: `feTurbulence`. This will form the basis of the paper grain texture.

```
// Initialise the filter.
let filter = svg.createFilter('rough-paper');

// Add turbulence to simulate paper grain.
filter.create('feTurbulence').set({
  type: 'fractalNoise',
  numOctaves: 5,
  baseFrequency: 0.04,
  seed: Gen.random(0, 100),
  result: 'turbulence'
});
```

The canvas should now be flooded with `feTurbulence`. What we'll do next is shine a light on it, specifically an `feDiffuseLighting` primitive with a nested `feDistantLight` node as the lighting type. This second node we can create directly after the diffuse lighting primitive, using SvJs method chaining.

```
// Shine diffuse lighting on the turbulence.
filter.create('feDiffuseLighting').set({
  surfaceScale: 1,
  diffuseConstant: 1.3,
  in: 'turbulence',
  result: 'lighting'
}).create('feDistantLight').set({
  azimuth: 180,
  elevation: 45,
});
```

We should now have a paper-like texture covering the canvas. Next, we'll rough up the edges of our source graphic by plugging some turbulence into an `feDisplacementMap` primitive.

```
// Distort the paper source graphic with turbulence.
filter.create('feDisplacementMap').set({
  in: 'SourceGraphic',
  in2: 'turbulence',
  scale: 25,
  result: 'distortion'
});
```

This gives us an outline of aging paper with worn edges, but with no texture. To re-introduce the result of the lighting effect, we need to do some compositing. With the lighting output as the foreground and rough-edged paper as the background, the `in` operator will achieve the effect we're looking for.

```
// Merge the lighting with the rough-edged paper.
filter.create('feComposite').set({
  in: 'lighting',
  in2: 'distortion',
  operator: 'in',
  result: 'composite'
});
```

With the shape now in place and the texture showing through, we have a believable rough and ripped paper texture. However, we're missing one final ingredient: the color. To remedy this, we'll blend the output of `feComposite` with the output of `feDisplacementMap` and use a `mode` of `multiply`. This should salvage our original parchment gradient.

```
// Re-introduce the parchment gradient.
filter.create('feBlend').set({
```

```
  in: 'composite',
  in2: 'distortion',
  mode: 'multiply'
});
```

Figure 8-13 shows the result. Not bad! While not quite impressive enough to stand on its own (in my opinion at least), with variations of this effect, you have a means of injecting additional visual interest into other generative compositions.

Figure 8-13. *Our rough and ready paper texture*

Generative Textures

The previous example lays the groundwork for what we'll try next. Simulating textures is all well and good, but for the next example, we'll go full generative, that is, relinquish some control and relish the randomness

of the results. In the process, we'll also demonstrate the use of specular lighting of the point variety, so you can see how it differs from diffuse, distant lighting.

Copy the template folder and call it `24-rocky-randomness`. Below the background, we'll set up our source graphic, which will be a simple circle. To this circle, we'll apply both a gradient and a filter.

```
// Create our source graphic.
svg.create('circle').set({
  r: 300,
  cx: 500,
  cy: 500,
  fill: 'url(#random-gradient)',
  filter: 'url(#rocky-randomness)'
});
```

The next step is to set up a linear gradient. For this, we'll use an array of three random colors, and we'll also randomize the gradient's rotation.

```
// A random colour array.
let colours = [
  `hsl(${Gen.random(0, 360)} 80% 80% / 0.75)`,
  `hsl(${Gen.random(0, 360)} 80% 80% / 0.75)`,
  `hsl(${Gen.random(0, 360)} 80% 80% / 0.75)`
];

// A gradient with a randomised rotation and array of colours.
svg.createGradient('random-gradient', 'linear', colours, Gen.
random(0, 360));
```

Now to the filter. As with our other examples, our first node will be an `feTurbulence` primitive, which will act as our primary textural source. The `numOctaves` will vary between `2` and `7`, which will allow for quite a degree of variation in the level of detail. The `baseFrequency` and `seed` we'll also randomize.

```
// Initalise the filter.
let filter = svg.createFilter('rocky-randomness');

// Create the primary turbulence.
filter.create('feTurbulence').set({
  type: 'turbulence',
  numOctaves: Gen.random(2, 7),
  baseFrequency: Gen.random(0.003, 0.01, true),
  seed: Gen.random(0, 1000),
  result: 'turbulence'
});
```

Now we have raw turbulence covering the canvas, and while it does vary on each refresh, it still feels a little too uniform. How can we create something more organic and a little less grid-like? Well, we could distort it with … more turbulence!

To do this, we'll set up another `feTurbulence` primitive, this time using `fractalNoise`. The `baseFrequency` we'll ramp up a little, but we'll keep `numOctaves` and the `seed` to a similar range. Then we'll plug both into `feDisplacementMap`, randomize the `scale` value (which determines the strength of the distortion), and set `R` and `G` as the x and y channel selectors (purely because I didn't really like the default output of alpha on this occasion).

```
// Set up another instance of turbulence.
filter.create('feTurbulence').set({
  type: 'fractalNoise',
  numOctaves: Gen.random(3, 7),
  baseFrequency: Gen.random(0.01, 0.07, true),
  seed: Gen.random(0, 1000),
  result: 'noise'
});
```

```
// Distort the first instance of turbulence with the second.
filter.create('feDisplacementMap').set({
  in: 'turbulence',
  in2: 'noise',
  scale: Gen.random(25, 40),
  xChannelSelector: 'R',
  yChannelSelector: 'G',
  result: 'distortion'
});
```

We have some more natural-looking noise now, so let's spin up some specular lighting and see how the noise looks when lit up. We'll utilize `Gen.random()` on most of the attributes, including the point light coordinates.

```
// Shine a specular point light on the distorted output.
filter.create('feSpecularLighting').set({
  in: 'distortion',
  surfaceScale: Gen.random(5, 30),
  specularConstant: Gen.random(2, 6),
  specularExponent: Gen.random(10, 25),
  result: 'lighting'
}).create('fePointLight').set({
  x: Gen.random([-50, 500, 1050]),
  y: Gen.random([-50, 1050]),
  z: Gen.random(50, 250)
});
```

For the `x` and `y` coordinates, you'll notice we're using `Gen.random()` to pick from an array of values rather than a range; if you inspect these values, you'll see that these combinations keep the lighting to the periphery of the canvas. This is mainly to prevent any direct shine (think of an unwelcome camera flash in a photograph). If you load up the sketch at this point, you should see some interesting generative terrain emerge. We're not done yet though.

Before we bring the terrain and the source graphic back together, we're going to soften the edges of the latter with some `feGaussianBlur`. We'll then use `feComposite` along with the `in` operator as we've done in previous sketches, before recovering the original gradient using another `feComposite` primitive, this time with the `atop` operator.

```
// Blur the source graphic.
filter.create('feGaussianBlur').set({
  in: 'SourceGraphic',
  stdDeviation: Gen.random(25, 50),
  result: 'blur'
});

// Bring the lit texture in via the blurred source graphic.
filter.create('feComposite').set({
  in: 'lighting',
  in2: 'blur',
  operator: 'in',
  result: 'comp1'
});

// Recover the original gradient.
filter.create('feComposite').set({
  in: 'blur',
  in2: 'comp1',
  operator: 'atop'
});
```

And now we're ready to see some results! Figure 8-14 shows a render I particularly liked, but the variation in this sketch is quite large so it's best to play around until you land on something you like.

Figure 8-14. *Specular random rockiness*

And that concludes our foray into filters. As I mentioned before, there's so much more to explore where filters are concerned, and there are quite a few primitives I didn't have the space to touch on. It definitely warrants (and rewards) further independent exploration.

Summary

Let's do a quick recap of what was covered in this our penultimate chapter:

- How filters are constructed
- Filter regions and the SvJs `createFilter()` method
- Basic primitives like `feGaussianBlur`, `feFlood`, and `feDropShadow`

- More complex primitives like `feComposite` and `feBlend` that tie other primitives together
- The power of the `feTurbulence` primitive and its creative possibilities
- Distorting other primitives using `feDisplacementMap`
- Lighting and how three dimensions can be simulated using bump maps
- Diffuse and specular lighting and the various light source types
- How turbulence, displacement, and lighting can be used together to simulate and generate textures

In the next chapter, we'll wrap things up and touch on some topics that can help take your generative art to the next level.

CHAPTER 9

The Generative Way

Congratulations for having come this far! We've covered a lot, so it's time to take stock, stretch out those mental muscles, and make sure we don't end this book with an injury. We'll wind down by reviewing each chapter in little chunks, and we'll wrap up by previewing some of the paths you might pursue next in your generative journey.

The Journey So Far

We began with an introduction to SVG and what sets it apart from raster formats like PNG and JPG. We then moved on to the SvJs library and learned how to set up a local front-end development environment. The first chapter closed with a bit of a code dump to serve up some visual inspiration, but it was a little light on explanation, so the following two chapters picked up the slack.

For those new to JavaScript and/or programming in general, Chapter 2 offered a primer, while Chapter 3 delved into the core functionality of the SvJs library. We explored how to create basic shapes and how to work with text, groups, gradients, and patterns, and we created some colorful compositions and optical illusions along the way.

With the introduction of randomness in Chapter 4, we were finally able to go full generative and create sketches with results not rigidly set in advance. We learned how to randomize numbers within a range, randomly select items from an array, and construct regular grids using that

D. Matthew, *Generative Art with JavaScript and SVG*, Design Thinking,
https://doi.org/10.1007/979-8-8688-0086-3_9

staple of generative art, the nested for loop. We extended our knowledge of SVG with masks and clip paths and played with Gaussian and Pareto probability distributions.

Noise was introduced in Chapter 5 as a way of injecting a more organic kind of variation into our sketches, and other useful generative functions were covered to help us constrain and map these noise values to more useful ranges. In Chapter 6, we tapped into the power of the path element and its arsenal of commands, and we learned how to create lines and curves of the quadratic, cubic Bezier, and elliptical arc variety.

Chapter 7 was all about making things move. We explored event listeners and user interaction and reviewed no less than four different ways of animating SVG. We covered basic collision detection and circular motion, before continuing on to filters in Chapter 8. There we focused on a subset of filter primitives of most immediate use to the generative artist, and we learned how to connect these primitives together to achieve a variety of different effects.

The Voyage Forward

So where do you go from here? Have we fully explored the field of generative art, mapped all the territory? Not in the slightest! We've only really circled the tip of the proverbial iceberg, hinting at the hulking structure beneath. So if you do want to dive a little deeper yourself, I'll tentatively suggest some further topics (with the caveat that the material we've covered so far could be expanded upon ad infinitum).

Trigonometry

I was only able to offer a very cursory treatment of trigonometry in Chapter 7, so if this was your first time encountering the likes of `Math.PI` and `Math.sin()`, don't expect these concepts to take root (squared or

otherwise … apologies, poor pun). Trigonometry is a topic that deserves a deep dive of its own, and like much of the material in this book, the more exposure and practice you get – or better yet, the more you play – the more comfortable you'll be.

That said, I do find that interactive visualizations are a great way to kickstart comprehension, so to that end, I've set up a sketch showing the relationship between circular motion and right-angled triangles (the very foundation of trigonometry), a freezeframe of which you can see in Figure 9-1.

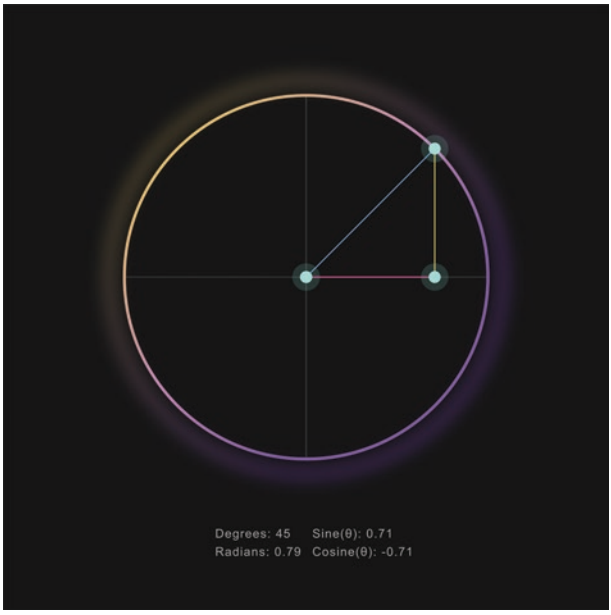

Figure 9-1. *An interactive trigonometry visualization*

As with the rest of the examples in this chapter, it's intended to illustrate rather than instruct, so the code isn't included here. You can, however, play with this particular example on the book's dedicated web page and inspect the code there if so inclined. I've also included further

reading and resources that should help you delve into the topic, including those I've relied upon myself over the years. These can be found at davidmatthew.ie/generative-art-javascript-svg/#trigonometry.

If you don't consider yourself to be of a particularly mathematical mindset, please don't be dissuaded from exploring this fascinating subject. You only need to be acquainted with a relatively small `subset` of it to tap into its creative possibilities, and this doesn't have to involve complicated equations.

Fractals

Another subject that doesn't necessitate complicated equations or mind-bending mathematical prowess is fractals. Yes, a fractal is a mathematical object, and some are enormously complex, but fractals in general are based on simple principles of recursion and self-similarity.

Recursion essentially means self-reference. A recursive function is one that continuously calls itself, an example of which we encountered in Chapter 7 with `requestAnimationFrame()`. Self-similarity, in relation to fractals, means that one part will resemble the whole, and vice versa. This is illustrated in Figure 9-2, which shows a famous fractal known as the Sierpinski triangle. As you can see, each building block is a reflection of the structure taken in its entirety.

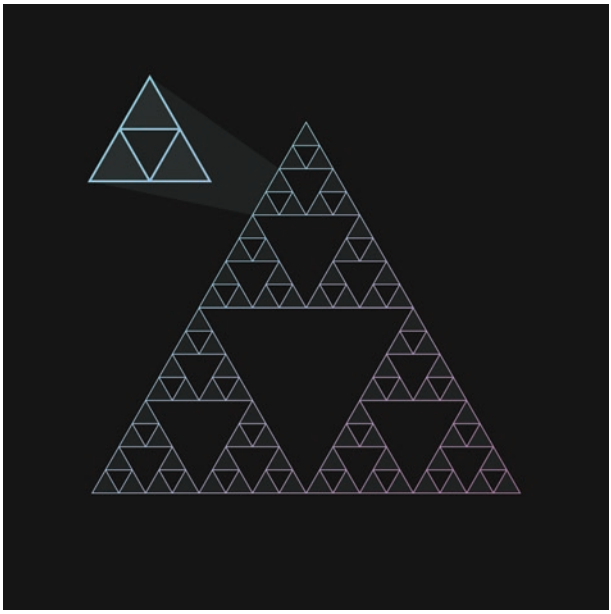

Figure 9-2. *Self-similarity in the Sierpinski triangle*

Where does recursion come in? Well, it can be found in every fractal, including the Sierpinski triangle, but to really illustrate the idea, let's take another well-known specimen, the Von Koch snowflake. The algorithm used to construct this fractal could be distilled down to the following (albeit simplified) steps:

1. Construct an equilateral triangle.
2. Divide each side into three segments.
3. Remove the central segment.
4. Repeat step 1.

The fourth step is what makes this algorithm recursive. Figure 9-3 shows what this process would look like after just three further iterations. In theory, these steps could continue on indefinitely, producing an ever-more intricate, endlessly magnifiable perimeter, which is why fractals

are sometimes seen as “windows into infinity.” In practice, iterations are usually limited in line with screen resolution or otherwise capped to prevent our browsers crashing and our processors crunching numbers they can no longer handle.

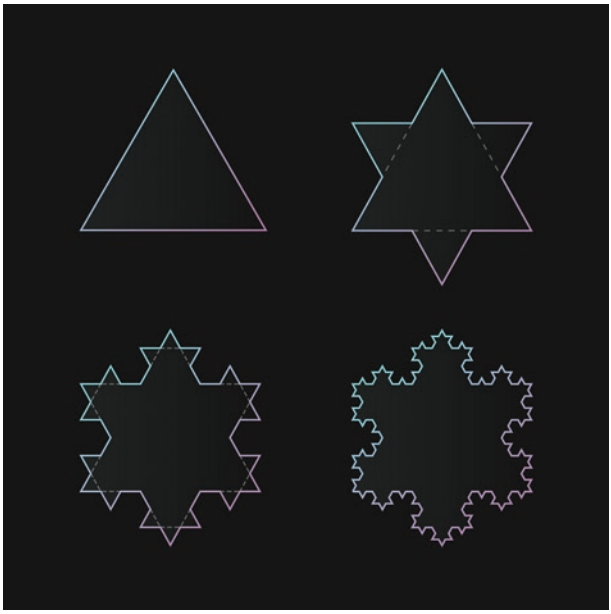

Figure 9-3. *Recursively constructing the Von Koch snowflake*

Fractals open up an entire universe of exploration and inspiration for the generative artist, far beyond what I’ve been able to hint at here. If your interest is piqued at this point, I’ve pooled together further resources at `davidmatthew.ie/generative-art-javascript-svg/#fractals` that I’d encourage you to explore.

Systems Simulations

From simple rules, unexpected complexity often emerges; this is not only true for fractals, but for a number of natural systems. Simulating such systems is usually the business of serious science, but in the more advanced areas of generative art, you'll see simulations of basic physics (think gravity, mass, density, etc.), particle systems (like fog, fire, and smoke), and organic behavior (from single cell interactions to the flocking patterns of birds).

Now, SVG isn't always the medium of choice for systems simulations – particle systems, for example, will perform better in pixel-based environments like the HTML canvas – but performance concerns aside, if you're looking to add an extra layer to your compositions, sprinkling a bit of science on top of the art, simulating natural systems can certainly be worth the work involved. A single sketch could become the basis of an endless stream of creative output, as much application as artwork.

John Conway's Game of Life is a good example of an organic simulation; it's a grid-based, zero-player game that simulates patterns of growth, decline, and evolution. Each cell in the grid can be in one of two states: "alive" or "dead." Living cells contain a fill color; dead cells do not. The rules that underpin the game are surprisingly simple, yet complexity can nonetheless emerge in the form of unexpected patterns and clusters of cells battling it out for survival. These rules are as follows:

1. Any living cell with either one or zero neighbors dies.
2. Any living cell with either two or three neighbors survives to the next "generation" (or iteration).
3. Any living cell with four or more neighbors dies.
4. Any empty (i.e., dead) cell with exactly three neighbors spawns to life.

Figure 9-4 shows a game a few generations in. Some clusters (like the square of cells in the top left and bottom right corners) are in stasis, meaning they survive but don't spread and thrive. Other cells are actively sprawling, exploring, disappearing, and re-spawning. It can be fun to watch!

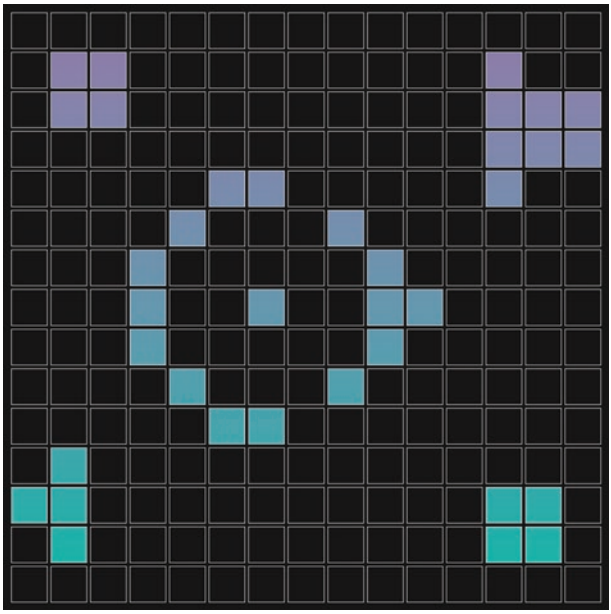

Figure 9-4. *A snapshot of a Game of Life in action*

I'll link to more examples for you to play with at `davidmatthew.ie/generative-art-javascript-svg/#systems-simulations`, including other kinds of simulations.

Closing Comments

At this stage, I've done enough summarizing, so I'd like to leave you with some (rather opinionated) parting tips to support you in your generative endeavors:

- Be comfortable never knowing enough. With any kind of coding, there's always so much more to know. And that's ok.
- Try to block off bits of time to play with code; you don't need to commit to all-nighters. Small habit-forming steps are more sustainable than the odd sprint here and there.
- Embrace simplicity; generative art doesn't have to be complicated to be good.
- Take inspiration from other artists; reverse-engineer examples that inspire you.
- Don't just copy; understand what you implement, and always put your own spin on things. It'll help you develop your own style.
- Give back. Get involved in the community. Contribute code, constructive critiques, or simply leave a positive comment on another artist's work and let them know you appreciate their efforts.
- And finally, appreciate your own efforts, regardless of the results. Output isn't everything, and your generative journey is your own.

Index

D. Matthew, *Generative Art with JavaScript and SVG*, Design Thinking,
https://doi.org/10.1007/979-8-8688-0086-3

F

G

H, I

J, K

L

M

N, O

P

Q

R

S

T, U

V

W, X, Y

Z